



INSTITUTO SUPERIOR DE POLITÉCNICO TECNOLOGIAS E CIÊNCIAS
DEPARTAMENTO DE ENGENHARIA E TECNOLOGIAS
CURSO DE ENGENHARIA INFORMÁTICA

Líria Djenaba Vilança Bá

20230237

RELATÓRIO DE TRABALHO PRÁTICO

Luanda

11/25/2025

INSTITUTO SUPERIOR DE POLITÉCNICO TECNOLOGIAS E CIÊNCIAS
DEPARTAMENTO DE ENGENHARIA E TECNOLOGIAS
CURSO DE ENGENHARIA INFORMÁTICA

RELATÓRIO DE TRABALHO PRÁTICO

Relatório técnico apresentado ao professor Daniel Sofrimento do Instituto Superior Politécnico De Tecnologia E Ciências, como requisito para avaliação.

Orientador: Daniel Sofrimento

DISCIPLINA	PROGRAMAÇÃO III
CURSO	ENGENHARIA INFORMÁTICA
TURMA	EINF5_M3

Luanda

11/25/2025

Sumário

1.	Introdução	1
2.	Justificação das Classes e Conceitos de POO	1
3.	Descrição dos Algoritmos Complexos	2
4.	Organização do Código	3
5.	Diagrama de Classes.....	4
6.	Conclusão	6

1. Introdução

Este trabalho prático teve como objetivo desenvolver uma aplicação em Java capaz de simular o processo de atendimento de clientes em um supermercado utilizando caixas de pagamento.

O sistema apresenta dois modos de funcionamento:

- **Modo Manual**, onde o utilizador controla diretamente as ações do sistema.
- **Modo Automático**, onde o sistema simula o processo continuamente, gerando chegadas de clientes, atendimentos e processamento temporizado.

O projeto reforça conceitos fundamentais de programação orientada a objetos, incluindo encapsulamento, abstração, modularização, manipulação de coleções e persistência em ficheiros.

2. Justificação das Classes e Conceitos de POO

Classes Implementadas

- **Cliente**
Representa cada cliente que chega ao supermercado. Possui identificador único e quantidade de produtos no carrinho.
- **Caixa**
Representa cada caixa de atendimento. Armazena a fila de clientes, controla o tempo restante do atendimento atual, contabiliza estatísticas e processa o tempo T de atendimento.
- **Mercado**
Gerencia o conjunto de caixas, controla a criação de clientes, distribuição automática na menor fila, remoção de caixas vazias e o processamento global da simulação.
- **GestorDeFicheiros**
Responsável pela gravação das simulações em ficheiros .txt, garantindo persistência de dados após o encerramento do programa.
- **ModoManual / ModoAutomatico**
Concentram o fluxo da simulação para cada tipo de interação definida pelo utilizador.

Justificação da Estrutura e POO

Embora o sistema não utilize herança entre as entidades centrais, ele aplica corretamente:

Encapsulamento

Cada classe controla seus próprios dados e expõe métodos de manipulação adequados.

Abstração

O utilizador interage com o sistema sem precisar conhecer detalhes de implementação — por exemplo, chamar `atenderEmTempoT()` sem saber como o cálculo interno é feito.

Polimorfismo (de uso estrutural)

Apesar de não ter métodos sobrepostos, o sistema aplica polimorfismo conceitualmente:

- Todas as caixas são manipuladas como objetos Caixa dentro de uma coleção.
- O comportamento de atendimento é uniforme para cada caixa, mas os resultados variam conforme o estado da fila.

Caso o sistema fosse expandido, seria possível implementar:

- estratégias diferentes de atendimento,
- hierarquia entre tipos de caixas,
- classes filhas para clientes especiais.

3. Descrição dos Algoritmos Complexos

a) Inserção de Cliente na Menor Fila

Quando um cliente é criado:

1. O mercado verifica todas as caixas existentes.
2. Seleciona a caixa com menor número de clientes (`getCaixaComMenosClientes`).
3. Adiciona o cliente nessa fila.
4. Se for o primeiro da fila, calcula o tempo de atendimento:

$$\text{tempoRestante} = \text{produtos} \times \text{tempoPorProduto}$$

Este mecanismo simula a escolha inteligente de fila.

b) Atendimento por Intervalo de Tempo T

O utilizador define um valor T e o sistema processa cada caixa da seguinte forma:

- Se $T < \text{tempoRestanteDoCliente}$, subtrai o valor e o cliente permanece.
- Se $T == \text{tempoRestante}$, o cliente é atendido e removido.
- Se $T > \text{tempoRestante}$, o cliente é removido, T é reduzido e o atendimento continua ao próximo cliente da fila.

Além disso:

- O total de clientes atendidos é incrementado.

- O tempo total de atendimento é acumulado.
- A média de atendimento é recalculada.

Esse processo permite que vários clientes sejam atendidos dentro de um único ciclo.

c) Modo Automático

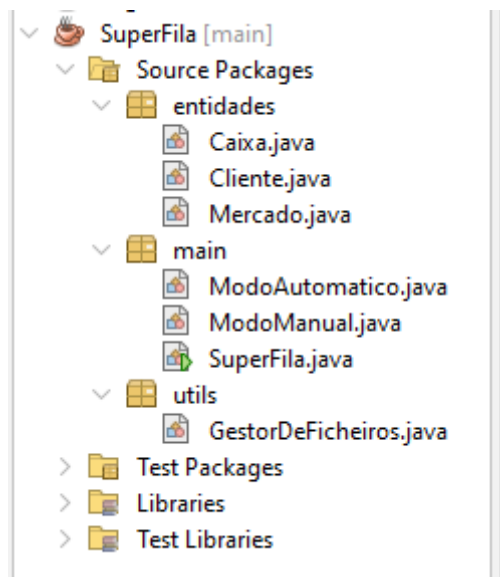
No modo automático:

- A cada passo da simulação:
 - O tempo avança 1 segundo.
 - A simulação atende os clientes existentes.
 - Um novo cliente é criado automaticamente após um intervalo aleatório.
- O utilizador pressiona **ENTER** para avançar o tempo ou 0 para terminar.

A cada ciclo, os dados são gravados no ficheiro de relatórios.

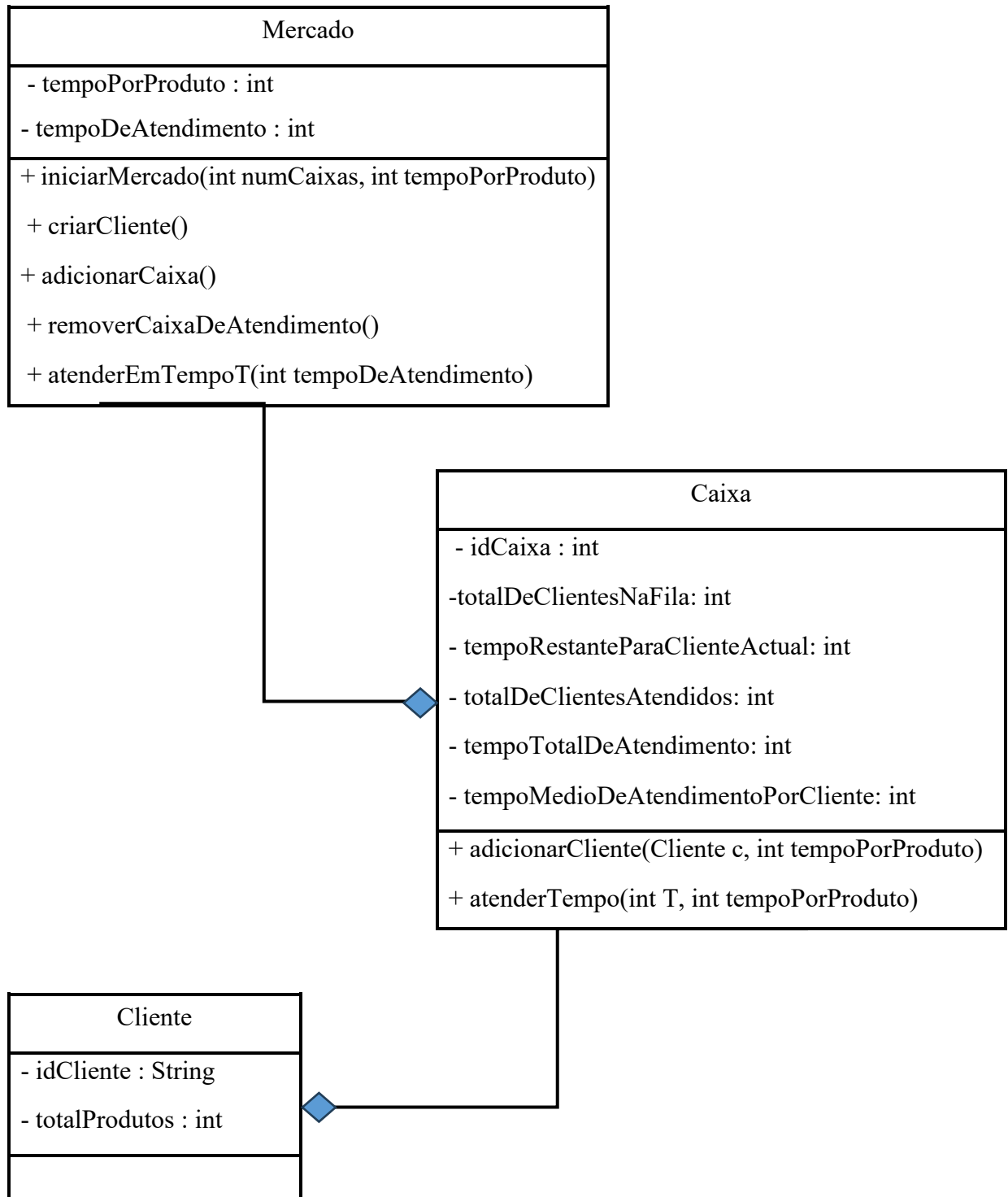
4. Organização do Código

O código foi dividido em pacotes e classes, seguindo boas práticas de modularização:



5. Diagrama de Classes

Abaixo está o diagrama de classes descritivo do sistema:



GestorDeFicheiros
- FICHEIRO: string
- FORMATO : DateFormatter
+ registrarSimulacao(String titulo, String conteudo)

6. Conclusão

O desenvolvimento deste trabalho permitiu simular com sucesso o fluxo de atendimento de um supermercado, aplicando conceitos essenciais de programação orientada a objetos, tais como modularização, encapsulamento e manipulação de coleções.

A implementação ofereceu:

- inserção de clientes em filas otimizadas,
- processamento realista de atendimento baseado em tempo,
- modos manual e automático de uso,
- persistência de dados através de ficheiros,
- visualização estruturada de filas e caixas.

O sistema, apesar de simples, demonstra como modelar situações reais com estruturas computacionais, podendo ser expandido futuramente para:

- relatórios gráficos,
- diferentes tipos de caixas,
- simulação estatística avançada,
- interface gráfica.

Cumpre assim os requisitos funcionais propostos e representa uma aplicação completa e totalmente funcional dentro do escopo da cadeira de Programação III.